Person A guide: baseline CNN, background bias, Grad-CAM

Written to you, Person A. Every part refers to the real file and function. Read with the code open.

0. Your part in one paragraph

You built and trained the **baseline**: a small CNN that starts from random weights and learns only from our PlantVillage subset. You evaluate it (accuracy, precision/recall/F1, confusion matrix). You run the **background-bias experiment**, which tests whether the model cheats by looking at the background. And you explain the models with **Grad-CAM** heatmaps plus our own **leaf-focus score**.

File: function	What it does	Why
models.py: conv_block, SimpleCNN	defines the baseline network	a small, readable 'no prior knowledge' model to compare with transfer learning
train.py: run_experiment, fit, train_one_epoch, evaluate_loss_acc	trains a model, keeps the best checkpoint, saves the history	one training loop for every experiment
evaluate.py: predict, compute_metrics, plot_confusion_matrix, evaluate_on_folder, evaluate_experiment	metrics and confusion matrices on PV test and PlantDoc	accuracy alone hides per-class problems
data.py: make_leaf_mask, random_background, LeafDataset (background=...)	leaf masks and the three background versions	the background-bias experiment
evaluate.py: evaluate_background_bias, background_table, plot_background_bias	tests every model on all backgrounds + PlantDoc	measures background reliance
gradcam.py: GradCAM, overlay, save_gradcam_examples	heatmaps of where the model looks	explainability
gradcam.py: mask_for_cam, leaf_focus_score, evaluate_leaf_focus	share of heatmap on the leaf	turns 'looks at the leaf' into a number

1. How a CNN works (the pieces you used)

Convolution and filters

A filter is a small grid of weights, e.g. 3×3. It slides over the image; at each position it multiplies its 9 weights (times 3 colour channels) with the pixels below and adds them up. The result is a **feature map** that is large where the pattern matches. Example: the filter [[-1,0,1],[-1,0,1],[-1,0,1]] responds to vertical edges. Our network **learns** the filter values. `nn.Conv2d(3, 32, 3, padding=1)` learns 32 filters on the colour image; `padding=1` keeps the height and width the same.

ReLU

ReLU(x) = max(0, x): negative values become 0. It adds the non-linearity that lets the network learn complex shapes. Without it, many layers would be no better than one.

Pooling

`nn.MaxPool2d(2)` keeps the largest value of each 2×2 window, so height and width halve. Example: [[1,3],[2,8]] becomes 8. Fewer numbers mean faster layers, and later filters 'see' a bigger part of the leaf.

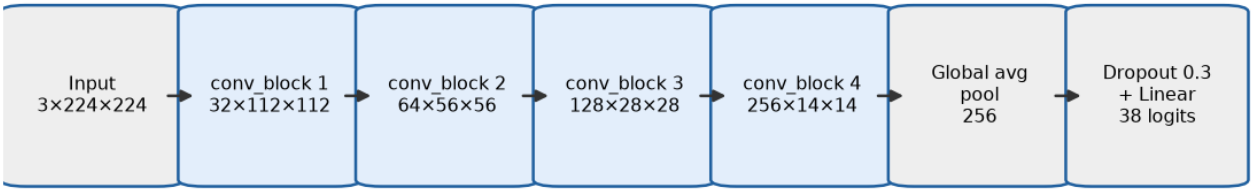
BatchNorm

`nn.BatchNorm2d` normalises every channel over the batch, then rescales it with learned parameters. This makes training faster and more stable.

Fully connected layer and softmax

After global average pooling, each of the 256 feature maps becomes one number. `nn.Linear(256, 38)` computes 38 weighted sums: one score (logit) per class. Softmax turns logits into probabilities: p_i = exp(z_i) / Σ exp(z_j). Example: logits (2, 1, 0) give probabilities (0.67, 0.24, 0.09). SimpleCNN outputs logits; the softmax happens inside CrossEntropyLoss during training, and via `torch.softmax` in `predict` (evaluate.py) and in the app.

2. Our baseline architecture, layer by layer



each conv_block = Conv3×3 -> BatchNorm -> ReLU -> Conv3×3 -> BatchNorm -> ReLU -> MaxPool 2×2 (halves height and width)

SimpleCNN (src/models.py). Shapes measured with a real forward pass.

Total parameters: **1,183,942** (counted by `count_parameters`). The table comes from running a 224×224 image through the real model:

Layer	Type	Output shape	Parameters
features[0][0]	Conv2d	32×224×224	896
features[0][1]	BatchNorm2d	32×224×224	64
features[0][2]	ReLU	32×224×224	0
features[0][3]	Conv2d	32×224×224	9,248
features[0][4]	BatchNorm2d	32×224×224	64
features[0][5]	ReLU	32×224×224	0
features[0][6]	MaxPool2d	32×112×112	0
features[1][0]	Conv2d	64×112×112	18,496
features[1][1]	BatchNorm2d	64×112×112	128
features[1][2]	ReLU	64×112×112	0
features[1][3]	Conv2d	64×112×112	36,928
features[1][4]	BatchNorm2d	64×112×112	128
features[1][5]	ReLU	64×112×112	0
features[1][6]	MaxPool2d	64×56×56	0
features[2][0]	Conv2d	128×56×56	73,856
features[2][1]	BatchNorm2d	128×56×56	256
features[2][2]	ReLU	128×56×56	0
features[2][3]	Conv2d	128×56×56	147,584
features[2][4]	BatchNorm2d	128×56×56	256
features[2][5]	ReLU	128×56×56	0
features[2][6]	MaxPool2d	128×28×28	0

Layer	Type	Output shape	Parameters
features[3][0]	Conv2d	256×28×28	295,168
features[3][1]	BatchNorm2d	256×28×28	512
features[3][2]	ReLU	256×28×28	0
features[3][3]	Conv2d	256×28×28	590,080
features[3][4]	BatchNorm2d	256×28×28	512
features[3][5]	ReLU	256×28×28	0
features[3][6]	MaxPool2d	256×14×14	0
pool	AdaptiveAvgPool2d	256×1×1	0
classifier[0]	Dropout	256	0
classifier[1]	Linear	38	9,766

Check one number by hand: the first conv has 32 filters of size 3×3×3 plus one bias each: 32 × (27 + 1) = 896 parameters. Most parameters sit in the last block (128→256 and 256→256 filters).

```
def conv_block(in_ch, out_ch):
    """Conv -> BatchNorm -> ReLU -> Conv -> BatchNorm -> ReLU -> MaxPool (halves H and W)."""
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),
    )


def forward(self, x):
    x = self.features(x)          # (B, 256, 14, 14)
    x = self.pool(x).flatten(1)   # (B, 256)
    return self.classifier(x)     # (B, num_classes) raw scores (logits)
```

Source: src/models.py: conv_block (line 14); src/models.py: SimpleCNN (line 27).

3. Training setup and why

Setting	Value	Where in the code	Why
Loss	CrossEntropyLoss	train.py: fit	standard loss for picking one of many classes; includes the softmax, so the model outputs raw logits
Optimizer	AdamW, weight decay 0.0001	train.py: fit	Adam adapts the step size per weight; the 'W' version applies weight decay correctly (regularisation)
Learning rate	0.001	notebook settings: CNN_LR	a common, safe start value for Adam on a small CNN
LR schedule	CosineAnnealingLR, T_max=12, stepped per epoch	train.py: fit	large steps first, small careful steps at the end
Batch size	64	notebook settings: BATCH_SIZE	fits easily in T4 memory; 64 images give a stable gradient estimate
Epochs	12	notebook settings: CNN_EPOCHS	keeps all runs within ~40 min on a T4; a scratch CNN needs more epochs
Model selection	best validation accuracy	train.py: fit	picks the epoch that generalises best without touching the test set
Seed	42	utils.py: set_seed (called in run_experiment)	same split, init and shuffling every run

Setting	Value	Where in the code	Why
Mixed precision	on GPU only (autocast + GradScaler)	train.py: train_one_epoch	about 2× faster on a T4 with the same accuracy
Input	3×224×224, ImageNet mean/std	data.py: get_transforms	same input for both models, so the comparison is fair

The real learning rate in each epoch (the same scheduler, recomputed here):

Epoch	1	2	3	4	5	6	7	8	9	10	11	12
SimpleCNN	1.0e-03	9.8e-04	9.3e-04	8.5e-04	7.5e-04	6.3e-04	5.0e-04	3.7e-04	2.5e-04	1.5e-04	6.7e-05	1.7e-05
EfficientNet	5.0e-04	4.7e-04	3.8e-04	2.5e-04	1.3e-04	3.3e-05						

How one training step works (`train_one_epoch`): take a batch of 64 images, run the model, compute the loss, `backward()` computes the gradient of the loss for every weight, `optimizer.step()` moves each weight a little in the direction that lowers the loss. One epoch = one pass over all training images.

```
for images, labels in loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()
    # Mixed precision (float16) on GPU: ~2x faster on a T4, same accuracy.
    with torch.autocast(device_type=device.type, enabled=scaler is not None):
        logits = model(images)
        loss = criterion(logits, labels)
    if scaler is not None:
        scaler.scale(loss).backward()  # scale loss so small float16 gradients don't become 0
        scaler.step(optimizer)
        scaler.update()
    else:
        loss.backward()
        optimizer.step()
```

Training augmentation for cnn_standard is 'standard': `Resize(size=(224, 224), interpolation=bilinear, max_size=None, antialias=True)`; `RandomHorizontalFlip(p=0.5)`; `RandomVerticalFlip(p=0.5)`; `ToTensor()`; `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`.

4. Overfitting and how we handled it

Overfitting = the model memorises the training images (including noise and background details) instead of learning general rules. Signs: training accuracy keeps rising, validation accuracy stalls or drops, and validation loss rises. Our ~7.4k training images are few for a CNN from scratch, so this is a real risk.

Measure	Where	How it helps
Data augmentation (flips; 'strong' in cnn_strong)	data.py: get_transforms	every epoch sees slightly different images
Dropout 0.3	models.py: SimpleCNN.classifier	randomly zeroes 30% of the 256 features during training, so the classifier cannot rely on one feature
BatchNorm	models.py: conv_block	stabilises training; also a mild regulariser
Weight decay 0.0001	train.py: fit (AdamW)	keeps weights small and simple
Global average pooling	models.py: SimpleCNN.pool	few parameters in the classifier (256×38 + 38)
Best checkpoint on validation accuracy	train.py: fit	we keep the epoch that generalises best (training still runs all epochs)

Look at `results/plots/history_cnn_standard.png`: left plot = loss, right plot = accuracy, train vs val.

[figure pending: `results/plots/history_cnn_standard.png` appears here after the Colab run]

5. Evaluation: accuracy, precision, recall, F1, confusion matrix

Toy example (not our data). 10 test images of class 'blight'; the model finds 8 of them, and it also calls 2 healthy leaves 'blight'.

- Recall(blight) = found / really blight = 8 / 10 = 0.80
- Precision(blight) = correct / predicted blight = 8 / (8 + 2) = 0.80
- F1 = 2·P·R / (P + R) = 0.80. F1 is high only if both are high.
- Accuracy = all correct predictions / all images (over every class).
- Macro average = plain mean over the classes, so every class counts equally (good for small classes).

`compute_metrics` (evaluate.py) uses scikit-learn's `classification_report` and saves accuracy, macro precision/recall/F1 and every per-class value into `results/eval/<name>_<set>_metrics.json`.

Confusion matrix (`plot_confusion_matrix`): rows = true class, columns = predicted class. We divide each row by its total, so each row sums to 1 and **the diagonal is the recall** of that class. A dark cell off the diagonal at (row X, column Y) means 'X is often mistaken for Y'. The extra **'other'** column counts predictions of a class outside the shown set; on PlantDoc that is one of the 10 PlantVillage classes PlantDoc does not have.

`evaluate_experiment` runs three tests per model: PlantVillage test (all 38 classes), PlantVillage test (only the 28 shared classes) and PlantDoc (28 classes). Domain gap = PV accuracy on the 28 classes − PlantDoc accuracy.

Results are not available yet

• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

Experiment	PV test acc (38)	PV macro-F1	PV acc (28 shared)	PlantDoc acc (28)	PlantDoc macro-F1	Domain gap
cnn_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_strong	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]

[figure pending: `results/eval/cnn_standard_pv_test_confusion.png` appears here after the Colab run]

6. The background-bias experiment

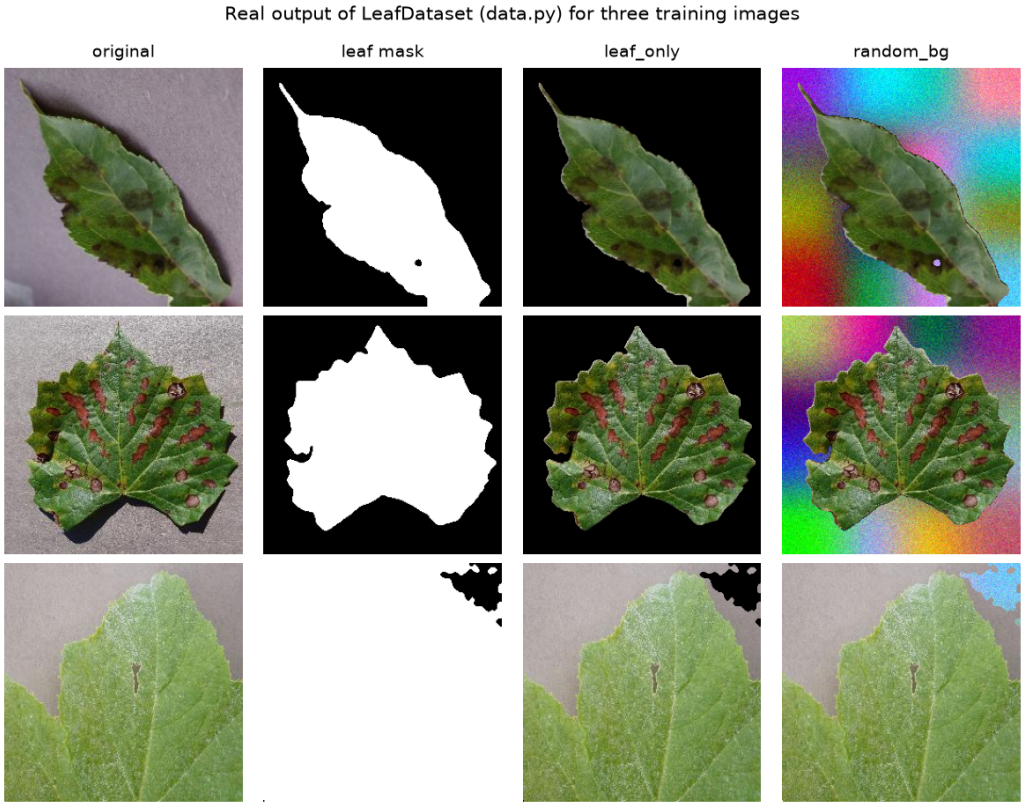
Question: does the CNN recognise the disease, or partly the plain lab background? Every PlantVillage photo has a similar background, so the background is a free clue. We cannot see that from accuracy alone, so we change only the background and watch what happens.

Training variant	What the training images look like	Code
cnn_standard	original photos	run_experiment(..., background='original')
cnn_leaf_only	background set to black, only the leaf left	background='leaf_only'
cnn_random_bg	leaf pasted on a NEW random colour-blob background each time it is loaded	background='random_bg'

How it works in the code: `LeafDataset.__getitem__` loads the photo and its mask, builds the new background (`Image.new('RGB', ...)` = black, or `random_background`), and combines them with `Image.composite(image, new_bg, mask)`: leaf pixels from the photo, everything else from the new background. This happens **before** the transforms, so augmentation still applies. Validation images use the same background as training (fixed seed), because val picks the best checkpoint. `random_background` blows up a random 4×4 grid of colours to full size and adds pixel noise (std 20).

```
if self.background != "original":
    mask = self.load_mask(idx)
    if self.background == "leaf_only":
        new_bg = Image.new("RGB", image.size)  # all black
    else:
        # np.random is seeded by set_seed / seed_worker, so training stays reproducible
        seed = (self.fixed_bg_seed + idx if self.fixed_bg_seed is not None
                else np.random.randint(2**31))
        rng = np.random.default_rng(seed)
        new_bg = random_background(image.size, rng)
    image = Image.composite(image, new_bg, mask)  # leaf where mask=255, new_bg elsewhere
```

`evaluate_background_bias` then tests each model on the same 2244 test leaves with all three backgrounds (random backgrounds fixed with seed 42, so every model sees identical images) and on PlantDoc. EfficientNet models are tested too, for comparison.



The three versions (real output of our code).

What we expected

- cnn_standard: good on 'original', worse on 'leaf only' and 'random bg' if it uses the background.
- cnn_random_bg: similar accuracy on all three backgrounds (it learned that the background is meaningless) and maybe a little better on PlantDoc.
- cnn_leaf_only: good on 'leaf only', but grey or random backgrounds are new to it, so it may drop there.

What we found

Results are not available yet

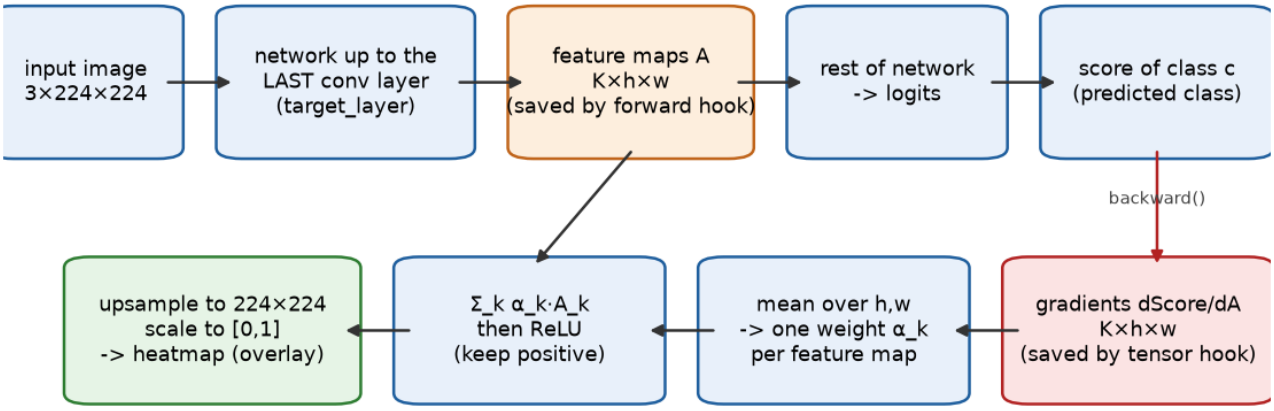
• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

Trained on	test: original	test: leaf only	test: random bg	PlantDoc
cnn_standard	[pending]	[pending]	[pending]	[pending]
cnn_leaf_only	[pending]	[pending]	[pending]	[pending]
cnn_random_bg	[pending]	[pending]	[pending]	[pending]
effnet_standard	[pending]	[pending]	[pending]	[pending]
effnet_strong	[pending]	[pending]	[pending]	[pending]

[figure pending: `results/plots/background_bias.png` appears here after the Colab run]

How to read it: compare along a row. A big drop from 'original' to 'random bg' means background reliance. Compare down a column to see which training helps on that kind of test. [Write your 2-3 sentence conclusion here after the Colab run.]

7. Grad-CAM step by step, in our code



leaf-focus score = Σ(heatmap × leaf mask) / Σ(heatmap)

Grad-CAM as implemented in GradCAM.__call__ (src/gradcam.py).

- **Target layer** (`get_gradcam_layer`, models.py): the last conv block, `features[-1]`. SimpleCNN gives 256 maps of 14×14; EfficientNet-B0 gives 1280 maps of 7×7.
- **Forward hook** `_save_activation` stores the feature maps A, and registers a tensor hook so that `_save_gradient` receives dScore/dA during backward().

- **Class score:** `logits[0, class_idx].backward()`, by default for the predicted class (raw logit, not softmax).
- **Weights:** `self.gradients.mean(dim=(2, 3))` averages each gradient map, giving one weight α_k per channel.
- **Weighted sum + ReLU:** `F.relu((weights * self.activations).sum(dim=1))` keeps only regions that push the class score up.
- **Upsample + scale:** `F.interpolate(..., mode='bilinear')` to 224×224, then min-max scaling to [0, 1].
- **Overlay** (`overlay`): jet colours (red = important) blended over the image with alpha 0.45.

```
logits = self.model(x)
probs = torch.softmax(logits.detach(), dim=1)[0]
if class_idx is None:
    class_idx = int(probs.argmax())  # explain the predicted class

self.model.zero_grad()
logits[0, class_idx].backward()  # gradient of this class score only

weights = self.gradients.mean(dim=(2, 3), keepdim=True)        # (1, K, 1, 1) = alpha_k
cam = F.relu((weights * self.activations).sum(dim=1, keepdim=True))  # (1, 1, h, w)
cam = F.interpolate(cam, size=image_tensor.shape[1:], mode="bilinear",
                    align_corners=False)[0, 0]
cam = cam - cam.min()
cam = cam / (cam.max() + 1e-8)  # scale to [0, 1]
```

`save_gradcam_examples` searches random images for 4 correct and 4 wrong predictions and plots image + overlay. On PlantVillage the title also shows the leaf-focus score.

[figure pending: `results/gradcam/cnn_standard_pv_test.png` appears here after the Colab run]

8. The leaf-focus score

Grad-CAM pictures are convincing but subjective. The leaf-focus score turns 'it looks at the leaf' into a number:

```
def leaf_focus_score(cam, mask):
    """Share of the Grad-CAM "heat" that lies on the leaf (0..1).

    score = sum(cam * mask) / sum(cam). 1.0 = all heat on the leaf,
    0.0 = all heat on the background. Compare it with the leaf's share of
    the image area: a heatmap spread evenly would score exactly that.
    """
    return float((cam * mask).sum() / (cam.sum() + 1e-8))
```

- `mask_for_cam` loads the leaf mask and resizes it to the heatmap size (224×224, nearest neighbour), giving 1 = leaf and 0 = background.
- Score = Σ(heat × mask) / Σ(heat): the share of all heat that lies on the leaf. 1.0 = all heat on the leaf, 0.0 = all on the background.
- `evaluate_leaf_focus` computes it for up to 300 random PlantVillage test images per model (seed 42), and also stores the mean **leaf area**, plus the averages for correct and wrong predictions.
- **Baseline to compare with:** a heatmap spread evenly over the image scores exactly the leaf's share of the area. So the score only means something if it is clearly **above** the mean leaf area.

Limits (say these yourself): the heatmap is coarse (14×14 or 7×7 upsampled), so heat spills over the leaf edge. The masks come from a simple threshold and are not perfect. The score cannot tell a lesion from a healthy part of the leaf. And it only works on PlantVillage,

because PlantDoc has no masks.

Results are not available yet

• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

Model	Leaf focus (mean)	Leaf focus (median)	Leaf area (mean)	Focus when correct	Focus when wrong
cnn_standard	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_strong	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_leaf_only	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_random_bg	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_standard	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_strong	[pending]	[pending]	[pending]	[pending]	[pending]

Things to double-check before the viva

These are places where the code is easy to misdescribe, or where something is limited or undocumented. Better to know them than to be surprised.

- The files now in `results/` come from a 1-epoch CPU smoke test on 2 images per class. They are NOT results. This builder detects that and prints [pending]. Run the Colab notebook, copy `results/` back, then run `python scripts/build_viva_pdfs.py` again.
- README.md says "see 'Getting it onto Colab' below", but that section does not exist in the README.
- Training on the command line (`python -m src.train`) uses batch size 32 and lr 1e-3 for BOTH models by default. The notebook uses batch 64, lr 0.001 (CNN) and 0.0005 (EfficientNet). Always quote the notebook values; those produced the results.
- `fit` (train.py) always runs ALL epochs and only keeps the checkpoint with the best validation accuracy. The code comment says "early-stopping style"; it is not real early stopping (training never stops early).
- The eval transform is `Resize((224, 224))`, which squashes non-square photos. PlantVillage photos are square (256×256), but PlantDoc photos are not (e.g. 512×384), so they get distorted. This is part of the measured domain gap.
- Leakage: our train and val share 0 leaf groups, but train and test share 2 leaf groups. These come from the authors' official split, which we use as is. Also, photos missing from the leaf map count as their own group, so the check cannot be 100% complete.
- Leaf masks: pixel = leaf if its brightest channel is > 25 in the segmented image, then a 5×5 median filter. The largest leaf share among our local masks is 1.00 (1.00 = the mask covers the whole image), so some masks are poor. Dark lesions or shadows can be misclassified.
- In `leaf_only` the background is solid black, which is itself one constant background. `random_bg` backgrounds are synthetic colour blobs, not real field scenes. Say this when you present the experiment.
- Leaf-focus score: the CAM is low-resolution (SimpleCNN 14×14, EfficientNet 7×7) and is blown up to 224×224, so heat spills over the leaf border. The score cannot tell lesion from healthy leaf, and it can only be computed on PlantVillage (PlantDoc has no masks).
- Grad-CAM's target layer for SimpleCNN is `features[-1]`, the whole 4th conv_block, including its MaxPool (output 256×14×14). Call it 'the output of the last conv block'.

- PlantDoc's spider-mites folder has only 2 images, and several PlantDoc classes are small, so per-class field numbers are noisy. The PlantVillage test set has at most 60 images per class (Potato healthy 32, Apple cedar rust 52).
- Each experiment is run once with seed 42. Differences of 1-2 points between models may be noise. We report no confidence intervals or repeated runs.
- How PlantVillage was collected comes from the dataset paper (Hughes & Salathé 2015; Mohanty et al. 2016), not from our code. Check the wording against the paper before quoting details.
- The "~40 min on a T4" estimate is not measured yet. Read `train_time_sec` in `results/history_*.json` after the run.

25 likely viva questions for you (with model answers)

Basic

Q1. What does a convolution layer do?

It slides small filters (3×3 in our SimpleCNN) over the image. At each position it multiplies the filter with the pixels under it and sums them, giving a feature map. Each filter learns to detect one pattern (an edge, a colour change, a spot). `nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)` in `conv_block`.

Q2. Why ReLU?

ReLU(x) = max(0, x). Without a non-linearity, stacked layers collapse into one linear function. ReLU is cheap and does not saturate for positive values, so gradients flow well.

Q3. What does max pooling do, and where is it in your model?

It keeps the largest value in each 2×2 window, halving height and width. It sits at the end of each conv_block, so the 224×224 input becomes 112, 56, 28 and then 14. It makes the network cheaper and a bit tolerant to small shifts.

Q4. Describe your baseline architecture in one breath.

4 conv_blocks (32, 64, 128, 256 filters), each with two 3×3 conv + BatchNorm + ReLU and a 2×2 max-pool. Then global average pooling to a 256-vector, Dropout 0.3 and a Linear layer to 38 logits. About 1.18M parameters.

Q5. What loss and optimizer did you use?

CrossEntropyLoss and AdamW (weight decay 0.0001), lr 0.001 with a cosine schedule, batch 64, 12 epochs (notebook settings; fit in train.py).

Q6. What is softmax, and where is it in your code?

It turns the 38 logits into probabilities that sum to 1. It is not a layer in SimpleCNN: CrossEntropyLoss applies it internally during training, and `predict` / GradCAM / app.py apply `torch.softmax` when we need probabilities.

Q7. What is the difference between accuracy and F1?

Accuracy = share of all images predicted correctly. F1 is computed per class as the harmonic mean of precision and recall; macro-F1 averages it over classes, so small classes count as much as big ones.

Q8. How do you read your confusion matrix?

Rows = true class, columns = predicted class, each row normalised to sum to 1. So the diagonal is each class's recall. Bright off-diagonal cells show which classes are confused. The extra 'other' column counts predictions outside the shown classes.

Intermediate

Q9. Why global average pooling instead of flatten + a big dense layer?

Flattening 256×14×14 = 50,176 values into a dense layer would need ~1.9M weights for 38 outputs, more than the whole network. GAP averages each feature map to one number (256 values), which keeps parameters low and reduces overfitting.

Q10. What does BatchNorm do?

It normalises each channel over the batch to mean 0 and std 1, then applies a learned scale and shift. Training becomes faster and more stable. In eval mode it uses running averages instead of batch statistics, which is why we call model.eval().

Q11. How did you prevent overfitting?

Flip augmentation, BatchNorm, Dropout 0.3, weight decay 0.0001, global average pooling (few parameters), and keeping the checkpoint with the best validation accuracy. We watch the train vs val curves in history_*.png.

Q12. How do you know whether the model overfits?

In the history plot: train accuracy keeps rising while val accuracy stalls or falls, and val loss goes up. A large gap between train and val accuracy is the warning sign.

Q13. Explain the three versions in your background-bias experiment.

Same training leaves, different backgrounds: original (plain lab background), leaf_only (background set to black using the mask) and random_bg (leaf pasted on a new random coloured-blob background every time it is loaded). All are tested on the same test leaves with all three backgrounds, and on PlantDoc.

Q14. How is the leaf mask made?

From PlantVillage's segmented photo of the same leaf (background painted black): pixel = leaf if its brightest channel > 25, then a 5×5 median filter (make_leaf_mask in data.py).

Q15. Walk me through Grad-CAM in your code.

A forward hook stores the last conv block's output A. We backpropagate the predicted class's logit; a tensor hook stores dScore/dA. We average the gradients over height and width to get one weight per channel, take the weighted sum of the maps, apply ReLU, upsample to 224×224 and scale to [0,1] (GradCAM.__call__).

Q16. What is the leaf-focus score?

sum(heatmap × leaf mask) / sum(heatmap): the share of Grad-CAM heat on the leaf. We average it over 300 random PlantVillage test images and compare it with the leaf's share of the image area, which is what an evenly spread heatmap would score.

Q17. Why the last convolutional layer for Grad-CAM?

It has the most high-level (class-specific) features, and it still keeps a spatial layout (14×14 for SimpleCNN). After global pooling the location information is gone.

Tricky

Q18. Why did you choose a small custom CNN instead of, say, VGG trained from scratch?

The baseline only has to show what a network learns from our 7,430 training images alone. A huge network from scratch would overfit and take too long on a free T4. A small, readable CNN is a fair 'no prior knowledge' baseline.

Q19. Your CNN might score very high on PlantVillage. Does that mean it understands plant disease?

Not necessarily. Lab photos are easy and uniform, and the model could use shortcuts like the background. That is exactly why we check field images (PlantDoc), background swaps and Grad-CAM leaf focus.

Q20. If accuracy drops a lot on random backgrounds for cnn_standard, what does it mean?

The only thing that changed is the background, so the model must have been using the background as evidence. Training with random_bg should make it rely on the leaf, which we would see as a smaller drop and higher leaf focus. (Check the real numbers in the table.)

Q21. Is a high leaf-focus score proof that the model is right for the right reason?

No. It only says the heat is on the leaf, not on the lesion. The heatmap is low-resolution (14×14 or 7×7) and blurred by upsampling, and the masks are imperfect. It is evidence, not proof.

Q22. Why might cnn_leaf_only do badly on original test images?

It only saw black backgrounds. A grey background is new to it, so the input distribution is different. A black background is also a constant clue, so it can be a shortcut too.

Q23. What would you do if validation accuracy was much lower than training accuracy?

Treat it as overfitting: more or stronger augmentation, more dropout or weight decay, fewer epochs (we already keep the best-val checkpoint), more data, or a smaller model. Also check for a bug, e.g. val images coming from a different distribution.

Q24. Why do you normalise inputs with ImageNet statistics in a CNN trained from scratch?

The model would also work with other statistics. We use the same input pipeline for both models so the only difference in the comparison is the model and its pretraining.

Q25. Grad-CAM uses the logit, not the softmax probability. Why?

The softmax probability of class c also depends on all other classes, so its gradient mixes in evidence against other classes. The raw class score (logit) gives cleaner per-class evidence, as in the original Grad-CAM paper. Our code calls `logits[0, class_idx].backward()`.